

Assignment 1

Introduction to Machine Learning

CS-3410 / MAT-3211, Spring 2026

Date: 24th February, 2026

Instructions

- The assignment is worth **100 marks**, and there are 2 parts to it.
 - Part A consists of theoretical problems, and you are expected to submit well organized LaTeX solutions on Google Classroom.
 - Part B consists of coding problems, and you need to attach a link to a GitHub repository containing the **jupyter notebook(s)**. The date of submission considered will be the date of the **last commit** to the repository. All code must be in Python, and the only libraries allowed are **numpy**, **datasets**, and **matplotlib** unless specified otherwise.
 - We encourage the use of LLMs and online resources provided that they are used to enhance your understanding and not to copy naively. You are required to disclose all sources you have used, which includes LLMs like ChatGPT, Gemini, etc.
-

Part A: Theoretical Problems (10 + 10 + 10 + 10 + 10)

1. Over the past 60 years, the number of marriages that took place each month, $x_i \in \mathbb{R}, i = 1, \dots, 720$ were recorded by the registrar. On the side, the total weight of flowers sold each month, $y_{i1} \in \mathbb{R}, i = 1, \dots, 720$ and the total weight of gold purchased each month $y_{i2} \in \mathbb{R}, i = 1, \dots, 720$ were recorded.
 - The flower sellers' association, having knowledge of only y_{i1} , uses the normal equations to obtain the coefficient θ_F of the linear regression $y_{i1} = \theta_F^T x_i + \theta'_F$.
 - The jewellers guild, on the other hand, has information about only y_{i2} and obtains the coefficient θ_J of the regression $y_{i2} = \theta_J^T x_i + \theta'_J$.
 - The jewellers and flower sellers meet the next week and create a new matrix Θ_{FJ} whose columns are θ_F and θ_J respectively.
 - You have complete information $\{(x_i, y_{i1}, y_{i2}) : i = 1, \dots, n\}$, and decide that you can use the normal equations to directly give you a regression matrix Θ such that if $y_i = (y_{i1}, y_{i2})$, then $y_i = \Theta^T x_i + \Theta'$.

Is $\Theta = \Theta_{JW}$? Suppose, furthermore, that instead of using the normal equations, all of you use gradient descent to calculate your respective optimizers. Is there a way to relate your algorithm to those of the jewellers and flower sellers' algorithms?

2. Solve the following problems in the field of kernel regression.

- (a) Given $K(x, y) = (x^T y + c)^l$ for any $l \geq 1 \in \mathbb{N}$, show that K is a valid kernel. What is the feature map ϕ corresponding to K ? (Hint : We discussed this in class for small values of l).
- (b) Prove that $K(x, y) = (x^T y)^2 - x^T y$ is not a valid kernel. (Hint : Contradict the conclusion of Mercer's theorem, assuming this is true)
- (c) We will now, in steps, show that $K(x, y) = \exp\left(-\frac{\|x-y\|_2^2}{2\sigma^2}\right)$ is a valid kernel.
 - i. Suppose that $K_1(x, y), K_2(x, y)$ are kernels with feature maps ϕ_1, ϕ_2 . Argue that $K_1(x, y) + K_2(x, y)$ and $cK_1(x, y)$ are also kernels for any $c > 0$, and find the feature maps corresponding to them.
 - ii. Argue that the product $K_1(x, y)K_2(x, y)$ is also a kernel. (You may use the Schur product theorem, <https://www.ethanepperly.com/index.php/2025/02/25/the-schur-product-theorem/>).
 - iii. Suppose that $K_n(x, y)$ is a sequence of kernels with feature maps ϕ_n such that $\lim_{n \rightarrow \infty} K_n(x, y) = K(x, y)$ for some function $K(x, y)$ and all x, y . Prove that $K(x, y)$ is a kernel, and find the feature map corresponding to it.
 - iv. Using this, argue that if $K_n(x, y)$ is a feature map, then so is $e^{K_n(x, y)}$. (Hint : $e^z = \sum_{n=0}^{\infty} \frac{z^n}{n!}$.)

Now, write

$$\exp\left(-\frac{\|x-y\|_2^2}{2\sigma^2}\right) = \exp\left(-\frac{\|x\|_2^2}{2\sigma^2}\right) \exp\left(-\frac{x^T y}{\sigma^2}\right) \exp\left(-\frac{\|y\|_2^2}{2\sigma^2}\right)$$

Use this to complete the proof. (Hint : let $f(z) = e^{-\|z\|_2^2/2\sigma^2}$ and use this to simplify your proof).

3. The odds of an event E is defined as the quantity $\frac{\mathbb{P}(E)}{1-\mathbb{P}(E)}$. A large number of unemployed people were profiled, and were asked for how long they were unemployed. Data of the form $(x_i, y_i)_{1 \leq i \leq N}$ was collected, where $x_i \in \mathbb{R}$ indicates the age at which they were fired, and $y_i \in \{0, 1\}$ asks whether they have been unemployed for more than a year or not. Logistic regression was performed, and a couple of interesting observations were made.

- If you were fired one year later than now, then your odds of being unemployed for more than a year went up by 1.75 percent.
- If you were fired at the age of 34, then your odds of being unemployed for more than a year was exactly 0.5.

Can you figure out the parameters of the logistic model? (Leave your answer as an algebraic expression, no need for a calculator)

4. Only people with specific height and weight characteristics are allowed to enroll for the army. While they haven't given a fixed rule, they have a dataset that looks something like this (here, data is "(height, weight, pass/fail)")

$$(175, 65, P), (158, 62, F), (188, 102, F), (176, 98, F) \\ (182, 87, P), (176, 52, F), (166, 56, P), (192, 104, P)$$

Your task is to create a decision tree fitting this data with zero classification error. However, you've been told that every decision can only be made on the basis of either height or weight (e.g. $h \leq 165$ or $w \geq 95$). What is the least number of nodes required to create a decision tree which fits the above data with zero classification error?

5. The following data set is provided to you, for classifying emails as spam or not spam. The first column has the word, the second the number of spam messages having that word, and the third the number of non-spam messages having that word.

<i>win</i>	20	15
<i>free</i>	40	15
<i>money</i>	30	15
<i>meeting</i>	0	60
<i>this</i>	30	45
<i>is</i>	20	30

You receive three separate emails, each with with the following contents.

- This is free money meeting.
- Free this, win money!
- Meeting is free.

For each of these messages, calculate the probability that they are spam, with and without using Laplace smoothing.

Part B: Coding Problems (20 + 20 + 10)

1. The goal of this question is to code a Random Forest from scratch and evaluate it on the diabetes dataset (available here).
 - (a) Write a Decision Tree class from scratch in Python. It should follow a well-defined schema, including:
 - A `build_tree(x: list[list[float]], y: list[float])` method that builds a decision tree using the greedy strategy discussed in class. This method should accept hyperparameters such as maximum samples per node and maximum height (which can be null/None).
 - A `predict(x: list[float]) -> float` method that takes a single data point and returns the predicted value.
 - (b) Use your Decision Tree class to implement a Random Forest class. This class should have a `build_forest` function that generates multiple trees. Each tree must be trained on a random subset of both features and samples, rather than the entire dataset.
 - (c) Run your code on the provided dataset to train both a single Decision Tree and your Random Forest model. Compare the predictions and performance of the two approaches.
2. In the lab, we trained a logistic regression model for predicting handwritten digits, but the gradient computation, gradient descent, and other optimizations were handled automatically using PyTorch. For this problem, you will build the learning algorithm manually:
 - (a) Load the **FashionMNIST** dataset.
 - (b) Implement **logistic regression** entirely from scratch. You must write your own functions to compute the gradients and perform gradient descent.
 - (c) Fit your model to the FashionMNIST training data using your custom implementation.
 - (d) Evaluate your model on the test set and clearly show its performance.
3. In this problem, you will explore both linear and non-linear regression techniques using the **diabetes dataset** introduced in question 1. (*Note: For this specific question, you are permitted to use `scikit-learn`*).
 - (a) Use `sklearn.linear_model.LinearRegression` to fit a standard ordinary least squares linear regression model to the training data. Compute and report its Mean Squared Error (MSE) on the test set.
 - (b) Now, use `sklearn.kernel_ridge.KernelRidge` to perform kernel regression. Fit the model using at least two different kernels. Clearly show the kernels used.